

Complete Architecture Plan: Building a Claude Code Alternative Using LM Arena

1. Objective

Build a fully autonomous coding agent that uses LM Arena as the model backend and replicates Claude Code functionality including:

- Autonomous coding
- File reading and writing
- Terminal execution
- Error fixing
- Multi-step reasoning
- Continuous context awareness
- CLI interface
- IDE integration
- Session persistence
- Near-zero failure operation

2. Core Design Principles

To achieve near-zero failure rate, the system must implement:

- Persistent browser sessions
- Automatic session recovery
- Response validation
- Retry logic
- Multi-layer fallback
- Structured parsing
- Local state management
- Error detection and correction loops

3. System Architecture

Main components:

1. Agent Core
2. LM Arena Connector (Playwright-based)
3. Context Manager
4. File System Interface
5. Execution Engine
6. Error Recovery System
7. CLI Interface
8. Memory Store

Flow:

User → CLI → Agent Core → LM Arena Connector → LM Arena → Response → Agent → Execute
→ Validate → Output

4. LM Arena Connector Design

Use Playwright persistent browser context.

Requirements:

- Persistent profile directory
- Automatic login retention
- Network monitoring
- Response extraction
- Timeout handling
- Auto refresh logic

Never use temporary sessions.

Always use persistent Chromium profile.

5. Agent Core Responsibilities

Agent must:

- Accept instructions
- Read local files
- Send structured prompt
- Receive response
- Validate response
- Execute code
- Handle errors
- Retry if needed
- Update context memory

6. Context Management

Maintain full conversation context locally.

Components:

- conversation_history.json
- file_index.json
- execution_log.json

This ensures continuity and prevents hallucination failures.

7. Failure Prevention Systems

Implement:

Layer 1: Response validation

Layer 2: Retry system

Layer 3: Session restart

Layer 4: Browser restart

Layer 5: Full recovery

Retry strategy:

Retry up to 3 times automatically.

8. File System Integration

Agent must support:

- Read files
- Write files
- Modify files
- Create projects
- Refactor code

9. Execution Engine

Capabilities:

- Execute Python
- Execute shell commands
- Capture output
- Capture errors
- Send errors back to model for fixing

10. Memory System

Store:

- conversation history

- execution history
- file history

This ensures continuity and stability.

11. CLI Interface

Example commands:

```
agent create app.py
agent fix errors
agent refactor project
agent build website
```

12. Recovery System

Automatic recovery steps:

1. Detect failure
2. Retry request
3. Refresh page
4. Restart browser
5. Restore session

13. Hardware Requirements

Minimum:
8GB RAM

Recommended:
16GB RAM
SSD storage
Modern CPU

14. Software Stack

Python 3.11
Playwright
Chromium
JSON storage

CLI framework

15. Expected Capabilities

Equivalent to Claude Code:

- Autonomous coding
- Project creation
- Debugging
- Refactoring
- Automation
- Continuous execution

16. Failure Rate Target

With proper implementation:

Expected failure rate: <1%

Recovery success rate: >99%